

A Modular DSP Architecture for Extreme-Precision Computation of π

José Ignacio Peinador Sala

Independent Researcher, Valladolid, Spain

[ORCID: 0009-0008-1822-3452](https://orcid.org/0009-0008-1822-3452)

joseignacio.peinador@gmail.com

June 21, 2026

Abstract

The calculation of π to extreme precision serves as both a benchmark for computational systems and a testbed for algorithmic innovation. Traditional approaches, based on the Binary Splitting evaluation of hypergeometric series, suffer from a *Memory Wall*: intermediate results grow to enormous sizes and saturate the memory bandwidth of modern multi-core architectures. We present the **Hybrid Stride-6** architecture, a *Shared-Nothing* design that exploits a formal isomorphism between modular arithmetic ($\mathbb{Z}/6\mathbb{Z}$) and polyphase decomposition in digital signal processing. By decomposing the Chudnovsky series into six independent modular channels, the architecture eliminates inter-thread synchronization and reduces memory contention. A key innovation, the *Stride-6 transition leaf*, aggregates blocks of six terms with exact phase correction, compressing the recursion tree depth by a factor of $\log_2 6 \approx 2.585$. The architecture is implemented in Python with the `gmpy2` multiple-precision arithmetic library. In a resource-constrained environment (Google Colab, 2 virtual CPUs, 12 GB RAM), it computes 10^8 digits of π in 19.9 minutes with 95% parallelization efficiency and a sustained speed of 83,729 digits per second, while using less than 7 GB of peak RAM. The modular decomposition introduces no numerical drift, as verified by exact agreement with reference digits. We compare the architecture with the state-of-the-art **y-cruncher** and discuss its applicability to other constants and hardware platforms.

Keywords: Pi Computation, Binary Splitting, Modular Arithmetic ($\mathbb{Z}/6\mathbb{Z}$), Polyphase Decomposition, Shared-Nothing Architecture, Parallel Computing, Multiple-Precision Arithmetic, Chudnovsky Series, Stride-6 Algorithm, Memory Wall.

1 Introduction

The quest for ever more digits of π has been a driving force in computational mathematics for centuries. Beyond the intrinsic mathematical interest, π calculation serves as a rigorous stress test for computer systems: it demands high-performance multiple-precision arithmetic, efficient memory management, and scalable parallelization. Modern algorithms, such as the Chudnovsky series evaluated via Binary Splitting [1], achieve

quasi-linear complexity $\mathcal{O}(N(\log N)^3)$ in the number of digits N . However, their practical performance on multi-core hardware is limited not by arithmetic throughput but by memory bandwidth—the so-called *Memory Wall* [2].

The root of the problem is that standard parallelization strategies for Binary Splitting attempt to parallelize a single, monolithic recursion tree. All threads must access and combine intermediate results stored in shared memory, creating contention on the memory bus and degrading scalability. Adding more cores yields diminishing returns because the memory bandwidth becomes the bottleneck.

This paper presents a fundamentally different approach. Instead of parallelizing the evaluation of a single series, we *decompose the series from the start* using the arithmetic structure of the terms. The decomposition is grounded in a recently established isomorphism between modular arithmetic on $\mathbb{Z}/6\mathbb{Z}$ and polyphase decomposition in multirate digital signal processing [4]. Under this isomorphism, the Chudnovsky series splits into six independent sub-series, each associated with a congruence class modulo 6. Because the sub-series operate on disjoint index sets, they can be computed in complete isolation—without any communication, synchronization, or shared memory access.

The resulting **Hybrid Stride-6** architecture embodies the *Shared-Nothing* paradigm [3]: six independent worker processes, each with its own memory space, compute their assigned sub-series using a modified Binary Splitting algorithm. The partial results are combined only at the end with a single rational addition. The architecture also introduces the *Stride-6 transition leaf*, which processes blocks of six terms at once with an exact phase correction, further reducing the recursion depth.

We demonstrate the effectiveness of this architecture through an extreme stress experiment: computing 10^8 digits of π on a severely resource-constrained platform (Google Colab with 2 virtual CPUs and 12 GB of RAM). The architecture completes the task in under 20 minutes with 95% parallelization efficiency, sustaining over 83,000 digits per second while using less than 7 GB of peak RAM. The result is verified against reference digits to confirm numerical integrity.

The paper is organized as follows. Section 2 reviews the necessary background on the Chudnovsky series and the polyphase isomorphism. Section 3 describes the Hybrid Stride-6 architecture in detail. Section 4 presents the experimental validation. Section 5 compares the approach with state-of-the-art alternatives. We conclude in Section 6.

2 Background

2.1 The Chudnovsky Series and Binary Splitting

The Chudnovsky series [1] provides the fastest known hypergeometric series for π :

$$\frac{1}{\pi} = 12 \sum_{k=0}^{\infty} (-1)^k \frac{(6k)!}{(3k)!(k!)^3} \frac{13591409 + 545140134k}{640320^{3k+3/2}}. \quad (1)$$

This series converges exponentially, producing approximately 14 decimal digits per term. Its evaluation to high precision is typically performed using the **Binary Splitting** algorithm, which recursively computes the partial sum as a rational number T/Q by splitting the summation range into halves, combining results via exact rational arithmetic, and avoiding premature floating-point rounding.

While Binary Splitting achieves quasi-linear complexity, its parallelization is hindered by the recursive dependence between sub-results. Standard multi-threaded implementations assign sub-trees to different threads, but the recombination step requires locks, shared memory, and the movement of large integer objects, leading to the Memory Wall.

2.2 The Polyphase Isomorphism

The theoretical foundation of our approach is the **Polyphase Isomorphism** between modular arithmetic and multirate signal processing [4]. The central result is:

Theorem 2.1 (Polyphase Isomorphism [4]). *Let $\{a_n\}_{n=0}^{\infty}$ be an absolutely convergent series. Define the discrete signal $x[n] = a_n$. Then, for any modulus $m \geq 2$, the modular projection*

$$S_r = \sum_{k=0}^{\infty} a_{mk+r}, \quad r = 0, \dots, m-1, \quad (2)$$

is exactly equivalent to the polyphase component $E_r(z) = \sum_k a_{mk+r} z^{-k}$ of the signal $x[n]$ evaluated at $z = 1$. The decomposition is a perfect reconstruction system:

$$S = \sum_{n=0}^{\infty} a_n = \sum_{r=0}^{m-1} S_r. \quad (3)$$

For $m = 6$, this decomposition has a special arithmetic significance: every prime $p > 3$ satisfies $p \equiv 1 \pmod{6}$ or $p \equiv 5 \pmod{6}$. The four remaining channels ($r = 0, 2, 3, 4$) contain only composite numbers (multiples of 2 or 3). The isomorphism thus partitions any arithmetic series into two *prime channels* and four *composite channels*, with no information leakage between them.

The crucial computational consequence is that the sub-series S_r are *independent*: their terms draw from disjoint index sets, requiring no shared state during computation. This independence is the key to the Shared-Nothing parallelization.

3 The Hybrid Stride-6 Architecture

3.1 Shared-Nothing Design

The Hybrid Stride-6 architecture applies Theorem 2.1 with $m = 6$ to decompose the Chudnovsky series (1) into six independent sub-series. Let a_k denote the k -th term of the series. The sub-series for residue r is:

$$S_r = \sum_{j=0}^{\infty} a_{6j+r}, \quad r \in \{0, 1, 2, 3, 4, 5\}. \quad (4)$$

Each sub-series S_r is assigned to an independent **Modular Worker**. The workers operate under a strict *Shared-Nothing* discipline:

- (i) **Isolated memory:** Each worker executes its own instance of the Binary Splitting algorithm, operating on its private address space. No memory is shared between workers.

- (ii) **No synchronization:** There are no locks, semaphores, or cache coherence protocols between workers during the computation phase. Workers start, compute, and terminate independently.
- (iii) **Optimal data locality:** The memory access pattern of each worker is local and predictable, maximizing L1/L2 cache utilization.

Once all workers finish, a lightweight *orchestrator* collects the six rational results $S_r = T_r/(P_r Q_r)$ and combines them with a single rational addition:

$$S = \sum_{r=0}^5 S_r = \sum_{r=0}^5 \frac{T_r}{P_r Q_r}. \quad (5)$$

3.2 The Stride-6 Transition Leaf

In standard Binary Splitting, a leaf node computes a single term a_k . In the Hybrid Stride-6 architecture, each worker processes indices of the form $n = 6j + r$, where j is the block index and r is the fixed residue. A naive leaf would still compute one term at a time. To reduce recursion overhead, we introduce the **Stride-6 Transition Leaf**, which aggregates a full block of six consecutive terms in one operation.

Algorithm 1 Stride-6 Transition Leaf (for a single residue r)

Require: Block index j (global), residue $r \in \{0, \dots, 5\}$.

Ensure: Compressed transition matrix (P, Q, T) for terms $k = 6j + r$ through $k = 6j + r + 5$.

```

1:  $k_{\text{start}} \leftarrow 6j + r$ 
2: Initialize  $P \leftarrow 1, Q \leftarrow 1, B_{\text{acc}} \leftarrow 0$ 
3: for  $m = 0$  to 5 do
4:    $k \leftarrow k_{\text{start}} + m$ 
5:   Compute Chudnovsky term components:  $P_k, Q_k, B(k) = 13591409 + 545140134 \cdot k$ 
6:    $P \leftarrow P \cdot P_k$ 
7:    $Q \leftarrow Q \cdot Q_k$ 
8:    $B_{\text{acc}} \leftarrow B_{\text{acc}} + B(k)$ 
9: end for
10:  $T \leftarrow Q \cdot B_{\text{acc}}$  ▷ Exact phase correction
11: return  $(P, Q, T)$ 
```

Remark 3.1 (Phase Correction). Line 8 of Algorithm 1 is the critical technical innovation of this work. In standard Binary Splitting for the Chudnovsky series, the recursion combines T terms using the rule $T_{ab} = Q_{mb}T_{am} + P_{am}T_{mb}$, where the linear term $B(k)$ is embedded in the T components. A naive implementation of a stride-6 leaf that multiplied individual T terms would introduce an “off-by-one-stride” phase error, because $B(k)$ depends linearly on k and does not factor across the block. Our solution directly accumulates $B(k)$ values across the six terms and synthesizes $T_{\text{leaf}} = Q \cdot B_{\text{acc}}$ at the end. This guarantees exact phase alignment with the original series at any scale.

The Stride-6 leaf reduces the recursion tree depth by a factor of $\log_2 6 \approx 2.585$: each leaf node processes 6 terms instead of 1, compressing the tree and minimizing function call overhead.

3.3 Implementation

The architecture is implemented in **Python**. Multiple-precision integer arithmetic is handled by the `gmpy2` library, which provides a C-level interface to the GMP (GNU Multiple Precision) library. Parallelism is achieved via the `multiprocessing` module: each Modular Worker runs as an independent Python process, bypassing the Global Interpreter Lock (GIL) and operating on its own memory space.

The workflow is:

1. The orchestrator creates 6 worker processes, each assigned a residue r and a range of block indices j .
2. Each worker executes Algorithm 1 within its own Binary Splitting recursion tree, producing three giant integers (P_r, Q_r, T_r) .
3. Workers terminate; the orchestrator collects results via inter-process communication.
4. The orchestrator computes $S = \sum_{r=0}^5 T_r / (P_r Q_r)$ using exact rational arithmetic.
5. The rational result (which equals $1/\pi$) is inverted and converted to decimal or hexadecimal format.

4 Experimental Validation: The 100M Barrier Run

4.1 Experimental Setup

To test the architecture under adverse conditions, we designed an extreme stress experiment: computing $N = 10^8$ (100 million) decimal digits of π on a severely resource-constrained platform. The experiments were conducted on **Google Colab**, a cloud-based Jupyter environment providing:

- 2 virtual CPUs (Intel Xeon at 2.2 GHz).
- 12 GB of total RAM.
- A virtualized Debian-based Linux environment.
- No GPU acceleration.
- No persistent disk (all I/O to temporary storage).

The Python implementation used `gmpy2` v2.1.5, Python 3.10, and the standard library `multiprocessing` module. The full source code and benchmark data are publicly available [6].

4.2 Results

Table 1 summarizes the results of the 100M Barrier Run.

Table 1: Benchmark results of the 100M Barrier Run (Google Colab, 2 vCPUs, 12 GB RAM).

Metric	Value
Digits calculated	100,000,000
Total time (including I/O)	1,194.32 s (19.90 min)
Pure parallel computation time	1,085.03 s (18.08 min)
Average sustained speed	83,729 digits/s
Speedup (vs. sequential on 1 core)	1.90×
Parallelization efficiency	95.0%
Peak RAM usage (total)	≈ 6.8 GB
Peak RAM per worker	≈ 1.1 GB
Modular orthogonality error	$\approx 0.00 \times 10^0$

The architecture achieved a sustained speed of over 83,000 digits per second, with a parallelization efficiency of 95% on two physical cores. The peak RAM usage remained below 7 GB, well within the 12 GB limit, demonstrating the memory efficiency of the Shared-Nothing design.

4.3 Integrity Verification

Result integrity was verified through three independent checks:

1. **Modular orthogonality:** The ℓ^2 norm of the terms in each modular channel was computed and summed. The difference between the sum of channel norms and the norm of the original series was numerically zero ($\approx 10^{-10^8}$), confirming that no information was lost or corrupted in the modular decomposition.
2. **Reference comparison:** The final decimal and hexadecimal sequences were compared against reference digits from the **y-cruncher** program [5] and public databases. All 10^8 digits matched exactly.
3. **Inverse check:** The computed value of $1/\pi$ was multiplied by the computed value of π to verify that the product was exactly 1 to within the working precision.

4.4 Scalability Discussion

The architecture scales linearly with the number of available cores, up to a maximum of 6 workers (one per modular channel). For systems with more than 6 cores, the work within each channel could be further parallelized using conventional sub-tree splitting, but without the Memory Wall penalty since each channel operates on its own memory space. This two-level hierarchy—inter-channel (Shared-Nothing) and intra-channel (shared memory)—is a natural extension of the architecture.

5 Comparison with State of the Art

Table 2 compares the Hybrid Stride-6 architecture with two alternatives: the standard monolithic Binary Splitting algorithm and the **y-cruncher** program [5], which holds the current world record for π computation.

Table 2: Philosophical and technical comparison of π computation architectures.

Characteristic	Monolithic BS	Hybrid Stride-6	y-cruncher
Data structure	Single recursion tree	6 independent trees	Hybrid, complex swap
Parallel model	Shared-memory threads	Shared-Nothing processes	Optimized threads + I/O
Memory pattern	Contiguous (saturates bus)	Per-core local (cache-friendly)	Sequential disk I/O
Scalability limit	Memory bandwidth	Up to $6\times$ cores	Disk speed
Implementation	Moderate	Moderate	Very high (ASM, AVX-512)
Portability	High	High (Python + gmpy2)	Low (platform-specific)

The Hybrid Stride-6 architecture occupies a unique niche: it prioritizes *resource efficiency* and *portability* over absolute speed. Unlike **y-cruncher**, which is written in highly optimized C++ with assembly-level intrinsics, our implementation uses pure Python with a C-backed arithmetic library. This makes it accessible for research, education, and deployment in heterogeneous environments where platform-specific optimizations are impractical.

6 Conclusion

We have presented the **Hybrid Stride-6** architecture, a novel approach to extreme-precision computation of π based on the polyphase isomorphism between modular arithmetic and multirate signal processing. The architecture achieves three key advances:

- (i) **Memory Wall mitigation:** By decomposing the series into six independent modular channels with isolated memory spaces, the architecture transforms a memory-bandwidth-limited problem into a CPU-limited problem with near-linear scalability.
- (ii) **Algorithmic innovation:** The Stride-6 transition leaf, with its exact phase correction, compresses the recursion tree depth and eliminates function call overhead while preserving arithmetic integrity at any scale.
- (iii) **Experimental validation:** The 100M Barrier Run demonstrates that the architecture computes 10^8 digits of π in under 20 minutes on commodity cloud hardware with 95% parallelization efficiency and less than 7 GB of RAM.

The modular approach is not limited to π . The polyphase isomorphism applies to any series whose terms can be indexed by integers, opening the door to efficient parallel computation of other mathematical constants (Catalan’s constant, Apéry’s constant, values of L -functions) and special functions.

Future work includes: (a) extending the architecture to distributed systems (MPI) for exascale computations; (b) implementing the Shared-Nothing design on FPGAs or ASICs as a “Hexa-Core Engine” with six physically isolated arithmetic cores and dedicated high-bandwidth memory; and (c) exploring the application of the modular filter to other hypergeometric series and modular forms.

Declarations

Funding: The author declares that no funds, grants, or other support were received during the preparation of this manuscript.

Author Contribution: J.I.P.S. is the sole author of this manuscript. The author independently conceptualized the architecture, designed the Hybrid Stride-6 algorithm and the Shared-Nothing parallelisation model, implemented the Python/gmpy2 prototype, conducted all benchmark experiments, and wrote the paper.

Ethics Declaration: Not applicable.

Competing Interests: The author has no relevant financial or non-financial interests to disclose.

Data Availability

The complete source code, benchmark scripts, and generated datasets for the Hybrid Stride-6 architecture are publicly available in the GitHub repository [Arquitectura-de-Hibridacion-Algoritmica-en-Z-6Z](#). An immutable archived version with a permanent Digital Object Identifier (DOI) has been deposited in the Zenodo platform. All underlying theoretical derivations and algorithmic details are explicitly described within the manuscript.

Acknowledgments

The author expresses deep gratitude to the open-source community for the computational and typesetting tools that facilitated the preparation of this manuscript. Special acknowledgment is given to the developers of Python, the `gmpy2` library (which provides the high-performance C backend for multiple-precision arithmetic), NumPy, and Matplotlib. The Google Colab platform is gratefully acknowledged for providing the cloud computing environment in which the 100M Barrier Run was executed. A sincere acknowledgment is also extended to the pioneers of the Binary Splitting algorithm and the Chudnovsky brothers, whose mathematical insights made extreme-precision π computation possible. Special thanks are directed to the anonymous peer reviewers and editorial staff, whose critical scrutiny, adversarial rigor, and constructive feedback are essential for maintaining the coherence and integrity of the scientific literature. This work is dedicated to the ongoing advancement of independent, foundational research in computational mathematics.

AI Use Statement

In accordance with modern scientific transparency standards, artificial intelligence models (specifically DeepSeek and Gemini) were utilized during the preparation of this manuscript. Their application was strictly limited to language editing, structural refinement, L^AT_EX formatting optimization, and acting as a critical analysis team (*red team*) to challenge, debate, and stress-test the theoretical arguments and logic. All core algorithmic designs,

the Stride-6 phase correction, the Shared-Nothing architecture, the benchmark implementation, and the performance analysis represent the original, independently verified intellectual work of the author.

Recommended Citation and License

Recommended Citation:

J.I. Peinador Sala, *A Modular DSP Architecture for Extreme-Precision Computation of π* , Zenodo (2026). <https://doi.org/10.5281/zenodo.XXXXXXX>

Open Access License:

This article is distributed under the terms of the Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)), which permits unrestricted use, distribution, and reproduction in any medium, provided you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license, and indicate if changes were made. The software is distributed under a dual license: PolyForm Noncommercial License 1.0.0 for academic and non-commercial use; a separate license is required for commercial applications.

References

- [1] D. V. Chudnovsky and G. V. Chudnovsky, *Approximations and complex multiplication according to Ramanujan*, in *Ramanujan Revisited: Proceedings of the Centenary Conference*, Academic Press, 375–472 (1988).
- [2] W. A. Wulf and S. A. McKee, *Hitting the memory wall: Implications of the obvious*, ACM SIGARCH Comput. Archit. News **23**(1), 20–24 (1995).
- [3] M. Stonebraker, *The case for shared nothing*, IEEE Database Eng. Bull. **9**(1), 4–9 (1986).
- [4] J. I. Peinador Sala, *Polyphase Isomorphism between Modular Arithmetic and Multi-rate Signal Processing*, preprint (2026). [10.5281/zenodo.17680023](https://doi.org/10.5281/zenodo.17680023)
- [5] A. Yee, *y-cruncher — A Multi-Threaded Pi Program*, (2023). [y-cruncher](https://y-cruncher.net/)
- [6] J. I. Peinador Sala, *Arquitectura de Hibridación Algorítmica en $\mathbb{Z}/6\mathbb{Z}$* , GitHub (2026). [Arquitectura-de-Hibridacion-Algoritmica-en-Z-6Z](#)